2. Operadores lógicos

En programación, muchas veces necesitamos evaluar **más de una condición** dentro de una misma estructura condicional. Para esto, utilizamos los **operadores lógicos**, que nos permiten **combinar múltiples condiciones en una sola expresión lógica**.

Estos operadores toman como entrada valores **booleanos** (**True** o **False**) y devuelven como resultado otro valor booleano.

2.1. Operador and

El operador **and** devuelve **True** solo cuando **todas las expresiones involucradas son verdaderas**.
Si al menos una de las condiciones es falsa, el resultado final será False.

```
print(True and True)
print(True and False)
```

✓ 0.0s

```
True
False
```

Este operador se utiliza cuando es necesario que todas las condiciones se cumplan al mismo tiempo.

2.2. Operador or

El operador **or** devuelve **True** cuando **al menos una de las expresiones es verdadera.**
Solo devuelve False si todas las condiciones son falsas.

```
print(False or True)
print(False or False)
```

✓ 0.0s

```
True
False
```

Este operador se utiliza cuando alcanza con que una sola condición sea verdadera.

2.3. Operador not

El operador **not** se utiliza para **invertir el valor lógico de una expresión booleana.**

```
activo = True
print(not activo)
```

✓ 0.0s

```
False
```

- Si la expresión original es **True**, el resultado será **False**.
- Si la expresión original es **False**, el resultado será **True**.

Operador	Descripción	Ejemplo
and (Y)	Devuelve True si **ambas condiciones** son verdaderas	(edad > 18 and registrado)
or (O)	Devuelve True si **al menos una** de las condiciones es verdadera	(usuario_es_admin or tiene_permisos)
not (NO)	Invierte el valor de una condición (True → False y viceversa)	not es_suscriptor

2.4. Combinación de expresiones booleanas

Los operadores lógicos se utilizan para combinar expresiones booleanas, que pueden ser valores directos o el resultado de comparaciones relacionales.

Si queremos, por ejemplo, determinar si un usuario puede acceder a un sistema:

- Debe tener más de **18 años**
- Debe estar **registrado**

Sin operadores lógicos, tendríamos que hacer algo como esto:

```
edad = 20
registrado = True

if edad > 18:
    if registrado:
        print("Acceso permitido")
```

En cambio, usando operadores lógicos, podemos escribirlo de manera más sencilla:

```python
if edad > 18 and registrado:
    print("Acceso permitido")
```

2.5. Uso combinado de operadores lógicos

Los operadores lógicos pueden combinarse entre sí y también con operadores relacionales, permitiendo construir expresiones más complejas.

```python
edad = 25
es_admin = False
tiene_permiso = True

print(edad >= 18 and (es_admin or tiene_permiso))
```

✓ 0.0s

```
True
```

Los paréntesis permiten controlar el orden de evaluación y mejorar la legibilidad de la expresión.

2.6. Evaluación perezosa (short-circuit evaluation)

Las expresiones lógicas en Python se evalúan utilizando evaluación perezosa, lo que significa que Python **deja de evaluar una expresión tan pronto como el resultado ya puede determinarse.**

Por ejemplo, en una expresión con or, si la primera condición es verdadera, Python no evalúa las siguientes, ya que el resultado final será True.

De manera similar, en una expresión con and, si la primera condición es falsa, el resultado final será False y el resto de las condiciones no se evalúan.

Este comportamiento mejora la eficiencia y evita evaluaciones innecesarias.